A Comparison between Deep Learning Models for Gene Prediction

Jason D. Shao[1], Zahra Tavakoli rad[2], Issac Xu[2], Thomas P. Trappenberg[2]

Abstract

Gene prediction is a challenging machine learning task that had been driven by Hidden Markov Models for more than two decades. Recently, gene prediction using only deoxyribonucleic acid (DNA) sequence has seen the incorporation of deep learning models that shows great promise, although their evaluations are typically biased towards mammals, and lacking resolution at the nucleotide level. One such tool based on long short-term memory (LSTM), Tiberius, is the hitherto best tool for this purpose. In this study, we evaluate the performance of Tiberius and our custom transformer tool on evolutionarily distant genomes at the nucleotide level. We find that our transformer-based model not only outperforms Tiberius in terms of accuracy and generalizability across non-mammalian genomes but also provides a finer resolution in identifying exon-intron boundaries and splice sites. Moreover, our tool demonstrates superior performance in distinguishing coding and non-coding regions, which is the primary focus of this project. These findings highlight the capability of transformer-based models to enhance gene prediction accuracy and improve the identification of functional genomic regions across diverse datasets.

1. Introduction

Gene prediction is the process of identifying regions of deoxyribonucleic acid (DNA) sequence in a genome that encode protein or ribonucleic acid (RNA) products. This process is of fundamental importance for any genomics project aiming to resolve novel genomes, as it forms the basis for nearly all downstream analyses. Despite recent advancements, machine learning approaches for gene prediction have not yet achieved satisfactory accuracy and often require extensive manual curation to ensure high-quality results.

Gene prediction typically involves using DNA genomic sequences to classify regions into coding and non-coding categories. This binary classification task, referred to as ab initio gene prediction when only DNA sequences are used, is particularly challenging due to the complexity of eukaryotic genomes and the lack of additional biological evidence for novel genomes.

Traditionally, ab initio gene prediction has relied on probabilistic modeling via Hidden Markov Models (HMMs), which predict genes by modeling their internal structures [2]. However, HMM-based approaches fail to capture the full complexity of eukaryotic genomes and are biased towards well-studied organisms like animals, plants, and fungi. As a result, they often perform poorly on evolutionarily distant genomes [8]. Recently, gradient-based deep learning has emerged as a promising solution. Tools like Helixer [10], which combine convolutional neural networks (CNNs) and long short-term memory (LSTM) architectures, marked significant progress in this domain. Tiberius [5], an advanced LSTM-based tool, further improved accuracy, outperforming both Helixer and HMM-based tools like Augustus [9]. These models leverage long-range interactions within DNA sequences to better predict gene structures. Studies such as Avsec et al. (2021) [1] have demonstrated that capturing such interactions significantly enhances prediction accuracy, underscoring the potential of models that focus on these dependencies. Building upon these insights, transformer-based architectures have emerged as a natural progression, offering superior capabilities for modeling long-range dependencies in complex genomic sequences.

Similarly, transformer-based architectures have shown promise in gene prediction, with DNA language models (LMs) like NT-MS [4] achieving notable results. NT-MS employs bidirectional encoder representations from transformers (BERT) and has been trained on multiple species. However, it

[1] Department of Biochemistry and Molecular Biology, Faculty of Medicine, Dalhousie University, Halifax, Canada
[2] Faculty of Computer Science, Dalhousie University, Halifax, Canada

underperforms compared to HMM-based tools like Augustus for ab initio gene prediction and has primarily been tested on human genomes [7].
In this study, we address these limitations by comparing the performance of Tiberius with our custom transformer model. Our model is specifically designed for the binary classification of coding and non-coding regions at the nucleotide level. By focusing on evolutionarily distant genomes, we aim to improve the precision, generalizability, and resolution of gene prediction tools, providing a robust framework for ab initio genome annotation.

2. Methods

2.1 Dataset

We used two high-quality genomes, Ergobibamus cyprinoides and Carpediemonas membranifera, from the phylum Metamonada, provided by Roger Lab. The genomes were in FASTA format, and annotations were in GFF3 format. Repeat regions were soft-masked using RepeatMasker [3] with the Dfam [11] database. These genomes were selected to evaluate the generalizability of Tiberius and our custom transformer across distantly related genomes. E. cyprinoides was used for training due to its reliable annotations, while C. membranifera was reserved for testing.

Table 1: Nucleotide counts for each genome, showing the distribution of protein-coding and non-protein-coding regions.

	E. cyprinoides	C. membranifera
Protein-coding Nucleotides	10788711	11617614
Non-Protein-coding Nucleotides	3630788	12620222
Total	14419499	24237836

2.2 Tiberius

Tiberius employs a CNN-LSTM architecture with an optional HMM layer (Fig. 1). For a fair comparison, the HMM layer was excluded during training. E. cyprinoides was used for training over 10 epochs with a sequence length of 9,999, and C. membranifera was used for testing. Outputs in GTF format were converted to binary coding/non-coding classifications using a custom Python script.

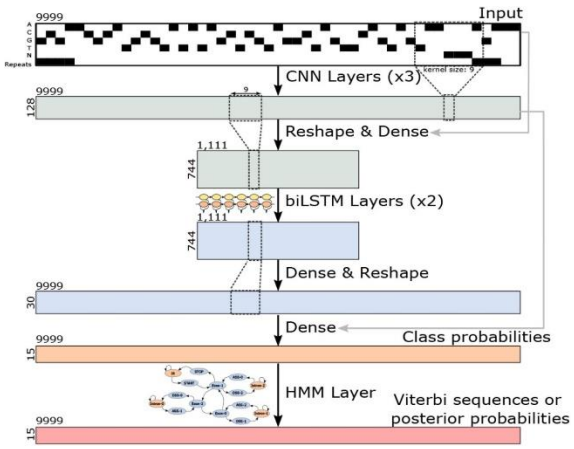

Fig. 1: The CNN-LSTM architecture of Tiberius (adapted from Gabriel et al., 2024). The HMM layer was detached during training.

2.3 Custom Transformer Model

LSTMs suffer from limitations such as forgetting (long-term dependency issues) and the vanishing gradient problem, making them less suitable for very long sequences. In the case of Tiberius, CNNs were used alongside LSTMs to enhance feature extraction and mitigate these issues. However, we opted for transformers due to their ability to overcome the forgetting problem and their exceptional performance in handling long sequences. Transformers effectively model long-range dependencies without the challenges faced by LSTMs, making them a superior choice for this task.

To fully leverage the capabilities of the transformer model, it was essential to prepare a well-structured dataset. We applied a sliding window approach where the window size was set to be smaller than the length of the shortest sequence in our dataset, approximately 5,000 nucleotides, ensuring comprehensive analysis of all sequences. Each window was designed with specific conditions at its start and end to accurately differentiate between coding and non-coding regions. To prioritize precision, these conditions were adjusted to favor true positives, resulting in stronger performance for predicting coding regions (1) compared to non-coding regions (0). A custom Python script was then used to convert multi-class predictions into binary labels, enabling thorough genome analysis and accurate sequence classification.

After preprocessing, the data was transformed into 14,408 features and 14,408 corresponding labels, ready for input into the model.

```
Processing windows for tig04970498:   0%|          | 0/1587 [00:00<?, ?window/s]
Processing windows for tig04970498:   6%|          | 94/1587 [00:00<00:01, 933.59window/s]
Processing windows for tig04970498:  13%|          | 200/1587 [00:00<00:01, 1004.18window/s]
Processing windows for tig04970498:  19%|          | 307/1587 [00:00<00:01, 1030.39window/s]
Processing windows for tig04970498:  26%|          | 417/1587 [00:00<00:01, 1057.71window/s]
Processing windows for tig04970498:  33%|          | 523/1587 [00:00<00:01, 1042.23window/s]
Processing windows for tig04970498:  40%|          | 628/1587 [00:00<00:00, 1043.51window/s]
Processing windows for tig04970498:  48%|          | 754/1587 [00:00<00:00, 1112.67window/s]
Processing windows for tig04970498:  55%|          | 866/1587 [00:00<00:00, 1101.42window/s]
Processing windows for tig04970498:  63%|          | 997/1587 [00:00<00:00, 1162.95window/s]
Processing windows for tig04970498:  70%|          | 1114/1587 [00:01<00:00, 1103.16window/s]
Processing windows for tig04970498:  77%|          | 1225/1587 [00:01<00:00, 1083.52window/s]
Processing windows for tig04970498:  84%|          | 1337/1587 [00:01<00:00, 1094.04window/s]
Processing windows for tig04970498:  91%|          | 1447/1587 [00:01<00:00, 1087.56window/s]
Processing windows for tig04970498:  98%|          | 1557/1587 [00:01<00:00, 1056.91window/s]
Processing sequences: 100%|          | 23/23 [00:16<00:00,  1.36sequence/s]
```

Fig. 2: Example of the sliding window technique applied to label DNA sequences.

2.4 Evaluation Metrics

We evaluated the models using standard metrics for binary classification:

$$Precision = \frac{TP}{TP + FP} \quad || \quad Recall = \frac{TP}{TP + FN} \quad || \quad F1 - score = 2 * \frac{Precision * Recall}{Precision + Recall}$$

Here, true positives (TP) and false negatives (FN) represent protein-coding nucleotides, while true negatives (TN) and false positives (FP) represent non-protein-coding nucleotides. Recall measures the proportion of

correctly identified coding nucleotides, precision measures the proportion of accurate coding predictions, and F1-score provides a harmonic mean of both. Metrics were calculated using a custom Python script.

3. Results

3.1 Evaluation of Tiberius

3.1 Evaluation of Tiberius

During training, Tiberius achieved an accuracy of 0.9213 and a loss of 4.1831 over 10 epochs (Fig. 2). On the test dataset, precision, recall, and F1-score were calculated, as summarized in Table 2. While the model performed well in identifying coding regions, challenges were observed in predicting non-coding regions with the same level of accuracy.

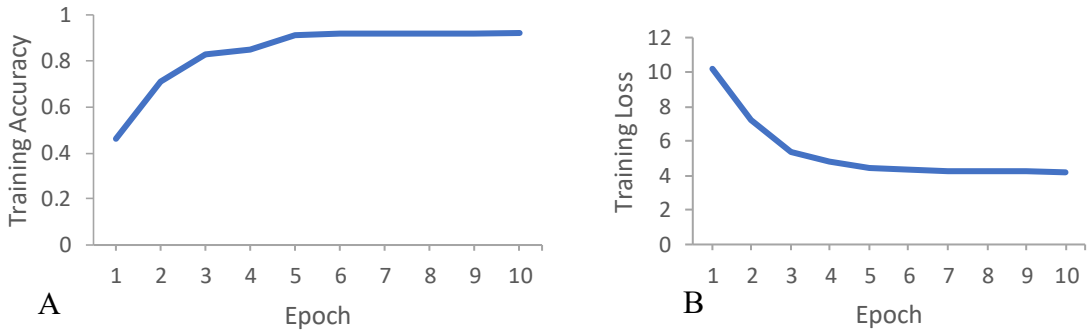

Fig. 2: Training accuracy (A) and loss (B) of Tiberius over 10 epochs.

3.2 Evaluation of CodingPT/Custom Transformer

The results of our custom transformer model highlight its capability to address the challenges of long sequences in genomic data. Unlike Tiberius, which struggled with non-coding regions, the transformer demonstrated superior performance in generalizing to these regions, achieving a higher F1-score and better recall for non-coding regions.

By leveraging the transformer's encoder module, we effectively captured long-range dependencies in the sequences without facing issues like vanishing gradients or forgetting, which are common in LSTM-based architectures. Since our task focused on binary classification rather than sequence generation, using only the encoder component allowed us to optimize feature extraction and sequence analysis.

The results showed that the transformer achieved an overall accuracy comparable to Tiberius for coding regions, with improvements in recall and precision for non-coding regions. These findings underscore the advantages of transformer-based architectures for complex genomic datasets.

3.3 Benchmarking

The performance of our custom transformer model was evaluated on C. membranifera after training on E. cyprinoides. During training, the model achieved a final accuracy of 97.4% and a validation accuracy of 98.0%, with the training loss decreasing to 0.049 by the last epoch (Fig. 3). This indicates effective learning from the training data.

```
Epoch 7/10: Train Loss: 0.072, Train Acc: 0.966, Val Loss: 0.061, Val Acc: 0.970
Training: 100%|          | 63/63 [00:15<00:00,  4.09batch/s]
Validation: 100%|          | 32/32 [00:01<00:00, 17.72batch/s]
Epoch 8/10: Train Loss: 0.061, Train Acc: 0.969, Val Loss: 0.067, Val Acc: 0.970
Training: 100%|          | 63/63 [00:15<00:00,  4.10batch/s]
Validation: 100%|          | 32/32 [00:01<00:00, 17.65batch/s]
Epoch 9/10: Train Loss: 0.051, Train Acc: 0.971, Val Loss: 0.049, Val Acc: 0.980
Training: 100%|          | 63/63 [00:15<00:00,  4.08batch/s]
Validation: 100%|          | 32/32 [00:01<00:00, 17.64batch/s]Epoch 10/10: Train Loss: 0.049, Train Acc: 0.974, Val Loss: 0.033, Val Acc: 0.980
```

Fig. 3: Training accuracy and loss for the custom transformer model on E. cyprinoides.

On the testing dataset, the model demonstrated strong performance for coding regions, achieving a precision of 0.97, recall of 0.98, and an F1-score of 0.97, indicating high reliability in predicting coding sequences. However, the performance for non-coding regions was lower, with a precision of 0.76, recall of 0.54, and an F1-score of 0.63. This outcome reflects the design of our sliding window conditions, which were intentionally adjusted to prioritize true positives for coding regions. While this approach improved the model's performance for coding regions, it resulted in a trade-off with lower performance for non-coding regions.

```
Classification Report:
              precision    recall  f1-score   support

  Non-coding       0.76      0.54      0.63      3752
      Coding       0.97      0.98      0.97     10223

    accuracy                           0.97     14408
   macro avg       0.87      0.76      0.80     14408
weighted avg       0.91      0.86      0.88     14408
```

Fig 4: Results of the tests ran on the *C. membranifera*

The table below compares the final results of Tiberius and our custom transformer model on the testing dataset, highlighting their respective strengths and weaknesses for coding and non-coding regions.

Table 2: Accuracy metrics for Tiberius and CodingPT/Custom Transformer on C. membranifera. Positive represents coding, and negative represents non-coding.

	Tiberius	CodingPT/Custom Transformer
Precision	0.7338	0.97
Recall	0.7142	0.98
F1-score	0.7239	0.97

These results highlight the model's strengths in accurately identifying coding sequences while emphasizing the challenges of capturing non-coding regions, possibly due to differences in training and test data characteristics. This provides a basis for future refinement of the model, particularly for improving generalization to diverse genomic datasets.

4. Discussion

The custom transformer model showed strong performance in predicting coding regions due to its ability to model long-range dependencies, outperforming Tiberius in resolving nucleotide-level boundaries. However, the sliding window approach, designed to prioritize true positives in coding regions, resulted in lower recall for non-coding regions. Additionally, dataset imbalance favored the coding class, limiting generalization to non-coding regions.

5. Conclusion

This study compared Tiberius and a custom transformer model for ab initio gene prediction, demonstrating the transformer's high accuracy for coding regions. However, challenges in generalizing to non-coding regions, driven by dataset imbalance and prioritization of coding regions, remain. While transformers hold great promise for gene prediction, future work should focus on balancing the dataset and refining the windowing strategy to enhance non-coding region predictions.

6. References

[1] Avsec, Ž., Agarwal, V., Visentin, D., Ledsam, J. R., Grabska-Barwinska, A., Taylor, K. R., Assael, Y., Jumper, J., Kohli, P., & Kelley, D. R. (2021). Effective gene expression prediction from sequence by integrating long-range interactions. Nature Methods, 18(10), 1196–1203. https://doi.org/10.1038/s41592-021-01252-x

[2] Burge, C., & Karlin, S. (1997). Prediction of complete gene structures in human genomic DNA. Journal of Molecular Biology, 268(1), 78–94. https://doi.org/10.1006/jmbi.1997.0951

[3] Chen, N. (2004). Using RepeatMasker to identify repetitive elements in genomic sequences. Current Protocols in Bioinformatics, Chapter 4, Unit 4.10. https://doi.org/10.1002/0471250953.bi0410s05

[4] Dalla-Torre, H., Gonzalez, L., Mendoza-Revilla, J., Lopez Carranza, N., Grzywaczewski, A. H., Oteri, F., Dallago, C., Trop, E., de Almeida, B. P., Sirelkhatim, H., Richard, G., Skwark, M., Beguir, K., Lopez, M., & Pierrot, T. (2024). Nucleotide Transformer: Building and evaluating robust foundation models for human genomics. Nature Methods, 1–11. https://doi.org/10.1038/s41592-024-02523-z

[5] Gabriel, L., Becker, F., Hoff, K. J., & Stanke, M. (2024). Tiberius: End-to-end deep learning with an hmm for gene prediction. bioRxiv. https://doi.org/10.1101/2024.07.21.604459

[6] Harada, R., Kume, K., Horie, K., Nakayama, T., Inagaki, Y., & Amagasa, T. (2023). Atlass: A scheme for end-to-end prediction of splice sites using attention-based bi-lstm. IPSJ Transactions on Bioinformatics, 16(0), 20–27. https://doi.org/10.2197/ipsjtbio.16.20

[7] Marin, F. I., Teufel, F., Horlacher, M., Madsen, D., Pultz, D., Winther, O., & Boomsma, W. (2023, October 13). Bend: Benchmarking dna language models on biologically meaningful tasks. The Twelfth International Conference on Learning Representations. https://openreview.net/forum?id=uKB4cFNQFg

[8] Scalzitti, N., Jeannin-Girardon, A., Collet, P., Poch, O., & Thompson, J. D. (2020). A benchmark study of ab initio gene prediction methods in diverse eukaryotic organisms. BMC Genomics, 21(1), 293. https://doi.org/10.1186/s12864-020-6707-9

[9] Stanke, M., Schöffmann, O., Morgenstern, B., & Waack, S. (2006). Gene prediction in eukaryotes with a generalized hidden Markov model that uses hints from external sources. BMC Bioinformatics, 7(1), 62. https://doi.org/10.1186/1471-2105-7-62

[10] Stiehler, F., Steinborn, M., Scholz, S., Dey, D., Weber, A. P. M., & Denton, A. K. (2021). Helixer: Cross-species gene annotation of large eukaryotic genomes using deep learning. Bioinformatics, 36(22–23), 5291–5298. https://doi.org/10.1093/bioinformatics/btaa1044

[11] Storer, J., Hubley, R., Rosen, J., Wheeler, T. J., & Smit, A. F. (2021). The Dfam community resource of transposable element families, sequence models, and genome annotations. Mobile DNA, 12(1), 2. https://doi.org/10.1186/s13100-020-00230-y